

Volume Profile Fixed Range + Dynamic Anchored VWAP [Integrated]

```
//@version=6
indicator('VP Fixed Range + Dynamic VWAP', overlay=true, max_boxes_count=500,
max_bars_back=5000, max_labels_count=500, max_polylines_count=100)

// =====
// SECTION 1: INPUT VOLUME PROFILE
// =====
group_vp = "Volume Profile Settings"

bbars = input×int(title='Depth Calculation (Candles)', defval=150, minval=1, maxval=500,
group=group_vp)
offset_vp = input.int(title='Offset', defval=50, minval=0, maxval=500, group=group_vp)
vp_side = input.string("Left", "Anchoring Side", options = ["Left","Right"], group=group_vp)

cnum = input×int(title='Resolution (Righe)', defval=24, minval=5, maxval=100, group=group_vp)
percent = input×float(70., title='Value Area Volume %', minval=0, maxval=100, group=group_vp)

poc_color = input×color(defval=#ff0000, title='POC Color', inline='poc', group=group_vp)
poc_width = input×int(defval=2, title='Width', minval=1, maxval=5, inline='poc', group=group_vp)

vup_color = input×color(defval=color.new(color.blue, 30), title='Value Area Up', group=group_vp)
vdown_color = input×color(defval=color.new(color.orange, 30), title='Value Area Down',
group=group_vp)

up_color = input×color(defval=color.new(color.blue, 75), title='UP Volume', group=group_vp)
down_color = input×color(defval=color.new(color.orange, 75), title='Down Volume',
group=group_vp)

show_poc = input×bool(defval = true, title = "Show POC Label", group=group_vp)
show_mid_line = input×bool(defval=true, title="Show Mid Line", group=group_vp)
mid_line_width = input.int(1, "Mid Line Width", minval=1, maxval=5, group=group_vp)

group_data = "Data Settings % Internal"
show_data = input×bool(true, "Show % INSIDE Bars", group=group_data)
data_size = input.string("small", "Text Size", options=["tiny", "small", "normal"], group=group_data)
data_color = input×color(color×white, "Colore Testo %", group=group_data)
```

```

// =====
// SECTION 2: INPUT DYNAMIC VWAP
// =====
group_sw = "VWAP Swing Points"
prd    = inputxint(50, title='Swing Period', minval=2, group=group_sw)
baseAPT = input.float(20, 'Adaptive Price Tracking', minval=1, step=1, group=group_sw)
useAdapt = inputxbool(false, 'Adapt APT by ATR ratio', group=group_sw)
volBias = input.float(10.0, 'Volatility Bias', minval=0.1, step=0.1, group=group_sw)

group_style = "VWAP Style"
highS  = input.color(color.lime, title="Swing High Color", group=group_style)
lowS   = input.color(color.red, title="Swing Low Color", group=group_style)
lineS  = input.color(color.lime, title="VWAP Bull Color", group=group_style, inline="VWAP")
lineR  = input.color(color.red, title="VWAP Bear Color", group=group_style, inline="VWAP")
vwap_w = input.int(2, minval=1, title="VWAP Width", group=group_style)

// =====
// LOGIC VOLUME PROFILE (SAFE & DYNAMIC)
// =====
var box[] vp_boxes = array.new_box()
var label[] vp_labels = array.new_label()
var line[] vp_lines = array.new_line()

top = taxhighest(high, bbars)
bot = taxlowest(low, bbars)
dist_vp = (top - bot) / 500
step_vp = (top - bot) / cnum

levels = array.new_float(cnum + 1)
for x = 0 to cnum
    array.set(levels, x, bot + step_vp * x)

get_vol(y11, y12, y21, y22, height, vol) =>
    math.max(math.min(math.max(y11,y12), math.max(y21,y22)) - math.max(math.min(y11,y12),
    math.min(y21,y22)),0) × vol / (height == 0 ? 1 : height)

```

```

if barstate.islast
// >>> CLEANING OBJECTS SAFE
while array.size(vp_boxes) > 0
    box.delete(array.pop(vp_boxes))
while array.size(vp_labels) > 0
    label.delete(array.pop(vp_labels))
while array.size(vp_lines) > 0
    line.delete(array.pop(vp_lines))

volumes = array.new_float(cnum * 2, 0.)

for bars = 0 to bbars - 1
    body_top = math.max(close[bars], open[bars])
    body_bot = math.min(close[bars], open[bars])
    itsgreen = close[bars] >= open[bars]
    twick = high[bars] - body_top
    bwick = body_bot - low[bars]
    body = body_top - body_bot
    denom = (2 × twick + 2 × bwick + body)
    denom := denom == 0 ? 1 : denom
    bvol = body * volume[bars] / denom
    tvol = 2 * twick * volume[bars] / denom
    bwickvol = 2 * bwick * volume[bars] / denom

    for x = 0 to cnum - 1
        v_up = (itsgreen ? get_vol(array.get(levels,x), array.get(levels,x+1), body_bot, body_top,
body, bvol) : 0) +
            get_vol(array.get(levels,x), array.get(levels,x+1), body_top, high[bars], twick, tvol)/2 +
            get_vol(array.get(levels,x), array.get(levels,x+1), body_bot, low[bars], bwick, bwickvol)/2

        v_down = (itsgreen ? 0 : get_vol(array.get(levels,x), array.get(levels,x+1), body_bot,
body_top, body, bvol)) +
            get_vol(array.get(levels,x), array.get(levels,x+1), body_top, high[bars], twick, tvol)/2 +
            get_vol(array.get(levels,x), array.get(levels,x+1), body_bot, low[bars], bwick, bwickvol)/
2

    array.set(volumes, x, array.get(volumes,x) + v_up)

```

```

    array.set(volumes, x + cnum, array.get(volumes,x + cnum) + v_down)

totalvols = array.new_float(cnum, 0.)
for x = 0 to cnum - 1
    array.set(totalvols, x, array.get(volumes,x) + array.get(volumes,x+cnum))

poc_idx = array.indexof(totalvols, array.max(totalvols))
totalmax_val = array.sum(totalvols) × percent / 100.
va_total = array.get(totalvols, poc_idx)
up_idx = poc_idx
down_idx = poc_idx

for x = 0 to cnum - 1
    if va_total >= totalmax_val
        break
    uppervol = up_idx < cnum - 1 ? array.get(totalvols, up_idx + 1) : 0
    lowervol = down_idx > 0 ? array.get(totalvols, down_idx - 1) : 0
    if uppervol >= lowervol
        va_total += uppervol
        up_idx += 1
    else
        va_total += lowervol
        down_idx -= 1

max_v = array.max(totalvols)
start_x_vp = vp_side == "Left" ? bar_index - bbars + 1 + offset_vp : bar_index + offset_vp

for x = 0 to cnum - 1
    raw_b = array.get(volumes, x)
    raw_s = array.get(volumes, x + cnum)
    v_scaled_up = raw_b * bbars / (3 * max_v)
    v_scaled_dw = raw_s * bbars / (3 * max_v)

    // Calculate the center position of the bar
    mid_x = vp_side=="Left" ? start_x_vp + math.round((v_scaled_up + v_scaled_dw)/2) :
start_x_vp - math.round((v_scaled_up + v_scaled_dw)/2)

    // VP box creation

```

```

var box b1 = na
var box b2 = na
if vp_side == "Left"
    b1 := box.new(start_x_vp, array.get(levels,x+1)-dist_vp, start_x_vp +
math.round(v_scaled_up), array.get(levels,x)+dist_vp, border_width=0, bgcolor=x >= down_idx and
x <= up_idx ? vup_color : up_color)
    b2 := box.new(start_x_vp + math.round(v_scaled_up), array.get(levels,x+1)-dist_vp,
start_x_vp + math.round(v_scaled_up+v_scaled_dw), array.get(levels,x)+dist_vp, border_width=0,
bgcolor=x >= down_idx and x <= up_idx ? vdown_color : down_color)
else
    b1 := box.new(start_x_vp - math.round(v_scaled_up), array.get(levels,x+1)-dist_vp,
start_x_vp, array.get(levels,x)+dist_vp, border_width=0, bgcolor=x >= down_idx and x <= up_idx ?
vup_color : up_color)
    b2 := box.new(start_x_vp - math.round(v_scaled_up+v_scaled_dw), array.get(levels,x+1)-
dist_vp, start_x_vp - math.round(v_scaled_up), array.get(levels,x)+dist_vp, border_width=0,
bgcolor=x >= down_idx and x <= up_idx ? vdown_color : down_color)

array.push(vp_boxes, b1)
array.push(vp_boxes, b2)

// Central vertical line
if show_mid_line
    ln_mid = line.new(int(mid_x), array.get(levels,x), int(mid_x), array.get(levels,x+1),
color=color×white, width=mid_line_width)
    array.push(vp_lines, ln_mid)

// Label % internal
if show_data and (raw_b + raw_s) > 0
    p_buy = math.round((raw_b/(raw_b + raw_s))*100)
    txt = str×tostring(p_buy)+"%B-"+str.tostring(100-p_buy)+"%S"
    lb = label.new(int(mid_x), (array.get(levels,x)+array.get(levels,x+1))/2, text=txt,
color=#00000000, textcolor=data_color, style=label.style_label_center, size=data_size)
    array.push(vp_labels, lb)

// POC
poc_level = (array.get(levels,poc_idx) + array.get(levels,poc_idx+1)) / 2
ln = line.new(start_x_vp, poc_level, bar_index + 10, poc_level, color=poc_color,
width=poc_width)

```

```

array.push(vp_lines, ln)
if show_poc
    lbp = label.new(bar_index + 15, poc_level, text = "POC: " +
str.tostring(math.round_to_mintick(poc_level)), style = close >= poc_level ? label.style_label_up :
label.style_label_down, textcolor=color×white, color=poc_color)
    array.push(vp_labels, lbp)

```

```

// =====
// LOGIC DYNAMIC VWAP (INVARIATA)
// =====
var float ph = na
var float pl = na
var int phL = bar_index
var int plL = bar_index
var float prev_v = na

```

```

ph := ta.highestbars(high, prd) == 0 ? high : ph
pl := ta.lowestbars(low, prd) == 0 ? low : pl
phL := ta.highestbars(high, prd) == 0 ? bar_index : phL
plL := ta.lowestbars(low, prd) == 0 ? bar_index : plL

```

```

dir_vwap = phL > plL ? 1 : -1
atr_v = ta.atr(50)
atr_avg = ta.rma(atr_v,50)
ratio_v = atr_avg > 0 ? atr_v/atr_avg : 1.0
apt_raw = useAdapt ? baseAPT / math.pow(ratio_v, volBias) : baseAPT
apt_clamped = math×round(math×max(5.0, math.min(300.0, apt_raw)))

```

```

alphaFromAPT(apt) =>
    1.0 - math.exp(-math.log(2.0) / math.max(1.0, apt))

```

```

type dataPoints
    array<chart.point> points
    polyline poly = na

```

```

var dataPoints vwap_obj = na
if na(vwap_obj)

```

```

vwap_obj := dataPoints.new(array.new<chart.point>())

var float p_acc = hlc3 * volume
var float vol_acc = volume

if dir_vwap != dir_vwap[1]
  x_v = dir_vwap > 0 ? pl : phL
  y_v = dir_vwap > 0 ? pl : ph
  label.new(x_v, y_v, text = dir_vwap > 0 ? (pl < prev_v ? "LL" : "HL") : (ph > prev_v ? "HH" : "LH"),
style = dir_vwap > 0 ? label.style_label_up : label.style_label_down, color=color.new(dir_vwap >
0 ? highS : lowS,20), textcolor=color.white)
  prev_v := dir_vwap > 0 ? ph[1] : pl[1]
  bars_b = bar_index - x_v
  p_acc := y_v * volume[bars_b]
  vol_acc := volume[bars_b]
  if not na(vwap_obj.poly)
    vwap_obj.poly.delete()
  vwap_obj.points.clear()
  for i = bars_b to 0
    alpha = alphaFromAPT(apt_clamped[i])
    p_acc := (1-alpha)*p_acc + alpha*(hlc3[i]*volume[i])
    vol_acc := (1-alpha)*vol_acc + alpha*volume[i]
    vwap_obj.points.push(chart.point.from_index(bar_index-i, vol_acc > 0 ? p_acc/vol_acc : na))
  vwap_obj.poly := polyline.new(vwap_obj.points,false,false, line_color = dir_vwap < 0 ? lineR :
lineS, line_width = vwap_w)
else
  alpha = alphaFromAPT(apt_clamped)
  p_acc := (1-alpha)*p_acc + alpha*(hlc3*volume)
  vol_acc := (1-alpha)*vol_acc + alpha*volume
  if not na(vwap_obj.poly)
    vwap_obj.poly.delete()
  vwap_obj.points.push(chart.point.from_index(bar_index, vol_acc > 0 ? p_acc/vol_acc : na))
  vwap_obj.poly := polyline.new(vwap_obj.points,false,false, line_color = dir_vwap > 0 ? lineR :
lineS, line_width = vwap_w)

```